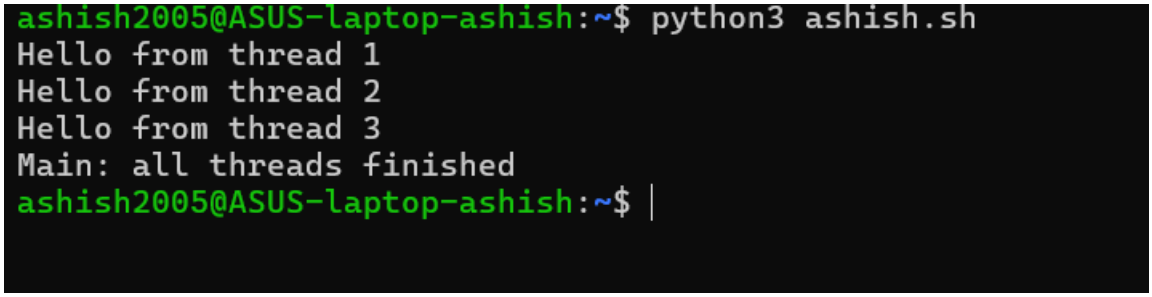
```
import os
import threading
import time


def program_1a():
    def worker(tid):
        print(f"Hello from thread {tid}\n", end="")

    threads = []
    for i in range(1, 4):
        t = threading.Thread(target=worker, args=(i,))  # create
        threads.append(t)
        t.start()                           # run

    for t in threads:
        t.join()                            # wait

    print("Main: all threads finished")
program_1a()
```

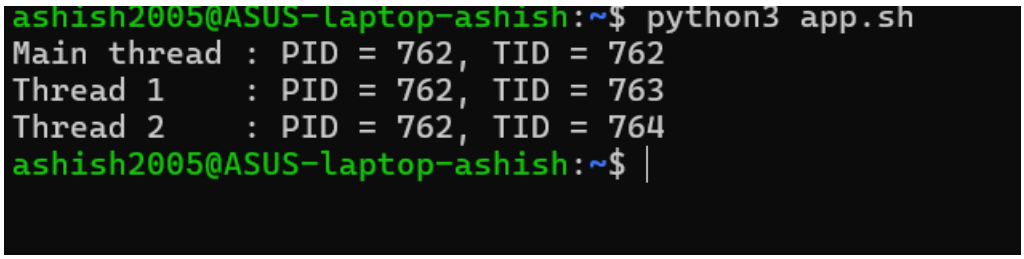



```
def program_1a():
    def worker(tid):
        print(f"Hello from thread {tid}\n", end="")
```

```
threads = []
for i in range(1, 4):
    t = threading.Thread(target=worker, args=(i,))  # create
    threads.append(t)
    t.start()                                # run

for t in threads:
    t.join()                                 # wait

print("Main: all threads finished")
```



```
  GNU nano 8.7.1                                                qwe.py
import os
import threading
import time
def program_1c():
    def download(n):
        print(f"Downloading file {n} ...\n", end="")
        time.sleep(1)                  # waiting, like the network
        print(f"File {n} done\n", end="")

    # 1) single thread
    start = time.time()
    for n in range(1, 5):
        download(n)
    single = time.time() - start

    # 2) four threads
    start = time.time()
    threads = []
    for n in range(1, 5):
        t = threading.Thread(target=download, args=(n,))
        threads.append(t)
        t.start()
    for t in threads:
        t.join()
    multi = time.time() - start

    print(f"Single thread : {single:.2f} s")
    print(f"4 threads     : {multi:.2f} s")
    print(f"Speed-up      : {single / multi:.2f}x")
program_1c()
```

```
ashish2005@ASUS-laptop-ashish:~$ nano qwe.py
ashish2005@ASUS-laptop-ashish:~$ python3 qwe.py
Downloading file 1 ...
File 1 done
Downloading file 2 ...
File 2 done
Downloading file 3 ...
File 3 done
Downloading file 4 ...
File 4 done
Downloading file 1 ...
Downloading file 2 ...
Downloading file 3 ...
Downloading file 4 ...
File 1 done
File 2 done
File 3 done
File 4 done
Single thread : 4.00 s
4 threads     : 1.00 s
Speed-up      : 3.99x
ashish2005@ASUS-laptop-ashish:~$
```

```
  GNU nano 8.7.1                                                    qwe.
import os
import threading
import time
def program_2a():
    balance = 0

    def deposit():
        nonlocal balance
        for _ in range(1000):
            temp = balance        # 1. read
            time.sleep(0)         #    thread switch can happen here
            balance = temp + 1    # 2. add and write

    threads = []
    for _ in range(5):
        t = threading.Thread(target=deposit)
        threads.append(t)
        t.start()
    for t in threads:
        t.join()

    print(f"Expected = 5000, Actual = {balance}")

program_2a()
```

```
ashish2005@ASUS-laptop-ashish:~$ python3 qwe.py
Expected = 5000, Actual = 1008
ashish2005@ASUS-laptop-ashish:~$
```

```
 GNU nano 6.2.1                                                    qwe.py
import os
import threading
import time
def program_2b():
    balance = 0
    lock = threading.Lock()       # the lock (mutex)

    def deposit():
        nonlocal balance
        for _ in range(1000):
            with lock:            # acquire ... release
                temp = balance
                time.sleep(0)
                balance = temp + 1

    threads = []
    for _ in range(5):
        t = threading.Thread(target=deposit)
        threads.append(t)
        t.start()
    for t in threads:
        t.join()

    print(f"Expected = 5000, Actual = {balance}")

program_2b()
```

```
ashish2005@ASUS-laptop-ashish:~$ python3 qwe.py
Expected = 5000, Actual = 5000
ashish2005@ASUS-laptop-ashish:~$
```

```
 GNU nano 8.7.1
import os
import threading
import time
def program_2c():
    slots = threading.Semaphore(2)

    def car(cid):
        print(f"Car {cid} is waiting\n", end="")
        slots.acquire()           # P(): take a slot, wait if 0
        print(f"Car {cid} PARKED\n", end="")
        time.sleep(1)             # parked for 1 second
        print(f"Car {cid} LEAVING\n", end="")
        slots.release()           # V(): free the slot

    threads = []
    for i in range(1, 6):
        t = threading.Thread(target=car, args=(i,))
        threads.append(t)
        t.start()
    for t in threads:
        t.join()

    print("All cars done")

program_2c()
```

```
ashish2005@ASUS-laptop-ashish:~$ python3 qwe.py
Car 1 is waiting
Car 1 PARKED
Car 2 is waiting
Car 2 PARKED
Car 3 is waiting
Car 4 is waiting
Car 5 is waiting
Car 1 LEAVING
Car 3 PARKED
Car 2 LEAVING
Car 4 PARKED
Car 3 LEAVING
Car 5 PARKED
Car 4 LEAVING
Car 5 LEAVING
All cars done
ashish2005@ASUS-laptop-ashish:~$
```